PROJETO G2 / MARCO 3 COMPLETO / ISSUES #16 A #23

Entenda o Marco 3.
Depois, teste o que aprendeu.

Caderno de estudo · SvelteKit e TypeScript · 22/10/2026

Todo o marco explicado, perguntas possíveis da professora e 24 questões de múltipla escolha. O gabarito comentado fica depois do simulado.

Abrir a versão HTML com correção automática

01 / O Marco 3 inteiro: cadastro e edição

O Marco 2 mostrava os dados. O **Marco 3 permite criar e editar** tarefas e projetos. Este guia cobre os oito itens da equipe, da #16 à #23, e mostra como eles se conectam no código.

Issue do Marco 3	O que você precisa entender
#16 · Ampliar o módulo da API (escrita)	Criar e atualizar tarefas e projetos com POST/PATCH e tipos TypeScript.
#17 · Criar o componente de formulário de tarefa	Organizar os campos em um componente com dados iniciais, opções da API e callback de envio.
#18 · Criar a tela de nova tarefa	Cadastrar tarefa em /tarefas/nova; ao vir de um projeto, sugerir esse projeto.
#19 · Criar a tela de edição de tarefa	Carregar a tarefa em /tarefas/:id/editar, preencher o formulário e salvar as alterações.
#20 · Validar o formulário	Validar campos, mostrar o erro junto deles e bloquear novos envios durante a gravação.
#21 · Tornar o formulário acessível	Usar labels, teclado, foco visível e mensagens que não dependem apenas de cor.
#22 · Criar cadastro e edição de projeto	Criar e editar projetos nas rotas /projetos/novo e /projetos/:id/editar.
#23 · Criar o slide de apresentação do Marco 3	Explicar a entrega e fazer uma demonstração em até 5 minutos.

1. A rota escolhe a página de cadastro ou edição.
2. A página prepara os dados; na edição, busca o registro existente.
3. O formulário recebe a digitação e confere o preenchimento.
4. Uma função TypeScript envia POST ou PATCH para a API.
5. Após a confirmação, a interface navega para a lista ou para o detalhe.

Base de estudo: código de `main` em 3edd465, consultado em 22/10/2026. Os exemplos são recortes comentados. As issues descrevem o pedido; o código mostra a implementação.

Uma diferença que vale saber explicar

A #17 pede reaproveitar o formulário de tarefa no cadastro e na edição. O componente `FormularioTarefa` foi preparado para isso, mas, nesta base, **a nova tarefa mantém formulário próprio e a edição usa o componente**. Ambos compartilham os auxiliares de validação. Nos projetos, cadastro e edição usam o mesmo `FormularioProjeto`.

A #19 ainda aparece aberta no GitHub na consulta de 22/10/2026, embora o arquivo de edição já exista em main. Explicar código existente e encerrar uma issue são ações diferentes.

02 / Onde encontrar o código

No VS Code, pressione **CmdP**, digite um dos caminhos abaixo e pressione Enter. O nome `+page.svelte` aparece várias vezes porque cada pasta representa uma página diferente.

Arquivo, a partir da raiz do projeto	Responsabilidade
`src/routes/tarefas/nova/+page.svelte`	Mostra o cadastro, guarda os valores preenchidos e coordena o envio.
`src/lib/servicos/api.ts`	Conversa com a API. Reúne as funções de leitura e as quatro funções de criação e atualização.
`src/tipos.ts`	Mantém o contrato de `Tarefa` e `Projeto` e define os tipos derivados usados para criar e atualizar.
`src/routes/+page.svelte`	É a lista, no endereço `/`. Tem a entrada para criar uma tarefa.
`src/routes/projetos/[id]/+page.svelte`	É o detalhe de um projeto. Seu link leva o ID do projeto até o cadastro.
`src/lib/componentes/FormularioTarefa.svelte`	Componente entregue pelo grupo, usado na edição. Recebe a regra de validação e a proteção contra envio repetido.
`src/lib/utils/validarObrigatorios.ts`	Regra da #20 para recusar texto obrigatório formado apenas por espaços, sem substituir a validação nativa.
`src/lib/utils/acessibilidadeFormulario.ts`	Auxiliar já entregue pelo grupo: mostra mensagens junto do campo e associa os erros por ARIA.
`tests/api.test.mjs`	Verifica o comportamento das funções da API, como método, endereço, corpo JSON e tratamento de erros.
`tests/api.integration.test.mjs`	Confere a API com operações HTTP e um banco de teste em memória.
`tests/validarObrigatorios.test.mjs`	Confere a regra de texto obrigatório, incluindo espaços, tabulações e texto válido.

Arquivo, a partir da raiz do projeto	Responsabilidade
`tests/api.types.ts` e `tests/tsconfig.json`	Conferem, com TypeScript, os tipos de entrada e retorno da API.
`src/routes/tarefas/[id]/editar/+page.svelte`	Busca a tarefa atual, passa os dados ao formulário e chama atualizarTarefa.
`src/lib/componentes/FormularioProjeto.svelte`	Formulário compartilhado para cadastrar e editar projetos; valida e coordena o envio.
`src/routes/projetos/novo/+page.svelte`	Define os valores vazios, acrescenta a data local e chama criarProjeto.
`src/routes/projetos/[id]/editar/+page.svelte`	Carrega o projeto e envia apenas os campos alterados.
`src/lib/utils/projetos.ts`	Define CamposProjeto e calcula hojeLocal para a data de criação.
`docs/marco-3/apresentacao_marco3.pptx`	Slides da equipe para a apresentação do Marco 3.

Um arquivo Svelte costuma juntar três partes:

- `<script lang="ts">`: TypeScript, com dados e funções.

- Marcação HTML/Svelte: os campos, textos e botões que aparecem.

- `<style>`: CSS, que organiza a aparência da página.

Os exemplos deste guia são recortes para estudar os conceitos; os arquivos do projeto são a referência para o código completo.

03 / Como o endereço encontra a página?

```
src/routes/tarefas/nova/+page.svelte
                    v
               /tarefas/nova

src/routes/tarefas/[id]/+page.svelte
                    v
           /tarefas/2, /tarefas/abc...
```

`nova` é uma parte fixa do endereço. `[id]` é uma parte variável. O SvelteKit escolhe a rota fixa antes da variável; por isso, abrir `/tarefas/nova` deve mostrar o cadastro, sem tentar buscar uma tarefa cujo ID seja “nova”.

E o projeto já selecionado?

O link do projeto leva uma informação extra depois do ponto de interrogação:

```
/tarefas/nova?projetoId=3
```

`/tarefas/nova` escolhe a página. `projetoId=3` sugere qual projeto selecionar. Isso é um **parâmetro de consulta**, também chamado de *query parameter*.

```
import { page } from '$app/state';

page.url.searchParams.get('projetoId'); // "3"
```

`page` vem do SvelteKit e informa a URL atual. Ele não é o roteador. Também não confundimos a consulta acima com `page.params.id`, que lê a parte `[id]` de uma rota dinâmica.

Antes de selecionar, o código confere se esse ID existe nos projetos que vieram da API. Se alguém escrever um ID inválido na URL, o formulário deve orientar a pessoa a escolher um projeto válido.

Por que usar `$derived` e `$effect` aqui?

O SvelteKit pode manter a mesma página aberta quando só a consulta muda. Por isso, observar a sugestão da URL permite atualizar o projeto ao ir de `?projetoId=1` para `?projetoId=2`. O efeito observa a sugestão e os projetos carregados; ele não deve observar o campo que a pessoa edita, para não desfazer a escolha manual dela.

As quatro páginas novas do marco

URL no navegador	Pasta dentro de src/routes
`/tarefas/nova`	`tarefas/nova/+page.svelte`
`/tarefas/2/editar`	`tarefas/[id]/editar/+page.svelte`
`/projetos/novo`	`projetos/novo/+page.svelte`
`/projetos/3/editar`	`projetos/[id]/editar/+page.svelte`

`:id` é uma forma de descrever o endereço no enunciado. A pasta do SvelteKit se chama `[id]`; no navegador você usa um valor real, como `2`. A regra de rota fixa também protege `/projetos/novo`.

04 / Como o formulário guarda o que eu digito?

Os dados enviados são: `projetoId`, `titulo`, `descricao`, `status`, `prioridade`, `responsavel` e `prazo`. Os nomes precisam respeitar o contrato que a API usa.

`NovaTarefa`: o formato dos dados

```
export type NovaTarefa = Omit<Tarefa, 'id'>;
```

`Omit` significa “usar os campos de `Tarefa`, retirando `id`”. O usuário não inventa o ID. O `json-server` o gera quando grava a tarefa.

O TypeScript ajuda a detectar nomes e tipos errados enquanto programamos. Ele não substitui as verificações do formulário nem a validação de um servidor real.

`$state`: um valor que atualiza a tela

```
let titulo = $state('');
```

Esse recorte começa com um título vazio. Quando o estado muda, o Svelte atualiza a parte da tela que depende dele. Na página completa, os campos podem estar agrupados em um objeto.

`bind:value`: ligar o campo ao estado

```
<label for="titulo">Título</label>
<input id="titulo" bind:value={titulo} required />
```

Ao digitar no campo, `titulo` recebe o valor. Se o código mudar `titulo`, o campo também muda. O `label` dá nome ao campo; o `for` aponta para seu `id`. `required` pede ao navegador que não envie esse campo vazio.

`onMount`: buscar as opções depois de abrir

`listarProjetos()` fornece as opções do seletor. `onMount` executa a busca quando a página é montada no navegador. Enquanto a resposta não chega, mostramos carregamento; se falhar, mostramos uma mensagem e permitimos tentar de novo.

Duas ideias diferentes: `onMount` inicia essa busca. O efeito acompanha a sugestão de projeto da URL. Um não substitui automaticamente o outro.

05 / Issue #17: o componente de formulário

Um componente é um pedaço da interface com seus próprios campos e comportamento. A página decide qual registro editar e o que fazer ao salvar; o formulário organiza a digitação.

Props são as entradas do componente

Em `FormularioTarefa.svelte`, `$props()` recebe estas entradas:

Prop	Para que serve
`valorInicial`	Dados de partida. Um objeto vazio serve como base de cadastro; uma tarefa existente preenche a edição.
`textoEnvio`	Texto do botão, como “Salvar alterações”.
`enviando`	Informa que a gravação está em andamento.
`erro`	Mensagem da falha de gravação, passada pela página.
`onenviar`	Função que recebe os campos preenchidos quando o formulário é enviado.

```
<FormularioTarefa
  valorInicial={tarefa}
  textoEnvio="Salvar alterações"
  {enviando}
  erro={erroEnvio}
  onenviar={salvar}
/>
```

Esse recorte é o uso real na página de edição. `onenviar={salvar}` entrega uma função. Quando o componente chama `onenviar(dados)`, a função `salvar` da página é executada. Isso é um **callback**.

Por que copiar os valores iniciais?

O componente começa com campos vazios e valores padrão e aplica `valorInicial` por cima. Usa `$state` para guardar a edição local e `untrack` na leitura inicial. Assim, a digitação é controlada pelo próprio formulário; receber os dados iniciais não significa alterar diretamente o registro original da API.

O seletor busca os projetos com `listarProjetos()` e repete as opções usando `{#each}`. O valor de cada opção é o ID do projeto, enquanto o texto mostrado é seu nome.

Conferência entre pedido e código: o componente permite receber dados diferentes, mas a tela `/tarefas/nova` ainda não o utiliza. Unificar as duas telas é uma melhoria possível; não apresente essa unificação como já feita. A descrição é obrigatória no cadastro e opcional no componente usado na edição.

Como chegar: abra `src/lib/componentes/FormularioTarefa.svelte`, procure `type Props`, `$props()`, `bind:value` e `onenviar`. Depois compare com a chamada na página de edição.

06 / O que acontece ao clicar em Salvar?

Um formulário trata seu envio com uma função. Ela impede o envio tradicional do navegador, prepara os dados e aguarda a API.

Operação	O que ela faz no projeto
`GET /projetos`	Lê os projetos para o seletor.
`POST /tarefas`	Envia os dados para criar uma tarefa.
`goto('/')`	Abre a página da lista depois da confirmação de criação.

Recorte do formato de uma requisição de criação:

```
fetch('http://localhost:3000/tarefas', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(dados)
});
```

- `method: 'POST'` informa que queremos criar um registro.

- `Content-Type` informa que o corpo usa JSON.

- `JSON.stringify(dados)` transforma o objeto TypeScript/JavaScript em texto JSON para enviar.

- `criarTarefa(dados)` reúne essa conversa com a API em uma função que a página chama.

Por que `async` e `await`?

```
const tarefaCriada = await criarTarefa(dados);
await goto('/');
```

A API leva algum tempo para responder. `await` espera o resultado dentro de uma função `async`. Só depois de criar a tarefa a navegação é iniciada.

A lista fica em `/`, e não em `/tarefas`. Ao voltar para ela, a busca da lista carrega os registros, incluindo a tarefa nova.

E se der erro?

`try` tenta a operação e `catch` trata uma falha. Se a gravação falhar, os campos continuam preenchidos para a pessoa tentar novamente. Durante o envio, o botão fica desabilitado para evitar pedidos repetidos.

Se a API já confirmou a criação, uma falha ao navegar não deve habilitar uma segunda gravação da mesma tarefa. A página reconhece o sucesso e permite voltar à lista. O `onDestroy` também marca quando você saiu desta tela, para uma resposta atrasada não forçar a navegação depois disso.

07 / A API agora também escreve

Antes, o módulo da API sabia ler: listar tarefas, buscar uma tarefa e carregar projetos. A #16 acrescenta quatro funções. A página passa os dados para elas; cada função monta a requisição correta e devolve a resposta da API.

Função e retorno	Pedido enviado
`criarTarefa(dados: NovaTarefa): Promise<Tarefa>`	`POST /tarefas`. Envia os campos da nova tarefa e recebe a tarefa criada, com ID.
`atualizarTarefa(id: string, dados: AtualizacaoTarefa): Promise<Tarefa>`	`PATCH /tarefas/ID`. Envia os campos que devem mudar e recebe a tarefa atualizada.
`criarProjeto(dados: NovoProjeto): Promise<Projeto>`	`POST /projetos`. Envia os campos do novo projeto e recebe o projeto criado, com ID.
`atualizarProjeto(id: string, dados: AtualizacaoProjeto): Promise<Projeto>`	`PATCH /projetos/ID`. Envia os campos que devem mudar e recebe o projeto atualizado.

Essas assinaturas são um mapa: o que está entre parênteses é a entrada; o que vem depois de `:` é o tipo do retorno.

`Omit`, `Partial` e `Promise<T>`

```
export type NovaTarefa = Omit<Tarefa, 'id'>;
export type AtualizacaoTarefa = Partial<NovaTarefa>;

export type NovoProjeto = Omit<Projeto, 'id'>;
export type AtualizacaoProjeto = Partial<NovoProjeto>;
```

- `Omit` retira um campo de um tipo. Na criação, retiramos `id` porque a API o gera.

- `Partial` torna os campos opcionais. Podemos atualizar só `status`, sem repetir o restante da tarefa.

- `Promise<Tarefa>` representa um resultado futuro. Depois do `await`, obtemos uma `Tarefa` se o pedido tiver sucesso. Uma falha precisa ser tratada.

São tipos derivados: os campos e os nomes do contrato original de `Tarefa` e `Projeto` continuam os mesmos. A tipagem auxilia o desenvolvimento, mas não valida sozinha os dados recebidos pela rede.

Quando usar GET, POST, PATCH ou PUT?

Método	Ideia e uso aqui
`GET`	Ler. Exemplo: carregar os projetos do seletor.
`POST`	Criar. Aqui, enviamos um novo registro à coleção `/tarefas` ou `/projetos`.
`PATCH`	Alterar parte de um registro existente. No json-server deste projeto, enviamos um objeto JSON com os campos que devem mudar.
`PUT`	Substituir a representação completa de um registro. Não é o método escolhido por estas funções, que aceitam atualizações parciais.

O formato aceito por um PATCH depende da API. Aqui usamos o contrato do json-server; isso não significa que todo servidor aceite o mesmo corpo JSON. A distinção entre alteração parcial e substituição é descrita na especificação de PATCH.

Atualizar uma tarefa: ID na URL, mudanças no corpo

```
const tarefa = await atualizarTarefa('2', {
  status: 'em-andamento'
});
```

O `'2'` identifica qual tarefa queremos alterar. O objeto informa o que deve mudar. O pedido corresponde a `PATCH /tarefas/2` com o corpo `{ "status": "em-andamento" }`. Os demais campos são preservados pelo json-server.

A função usa `encodeURIComponent(id)` ao montar o endereço. Assim, caracteres especiais do ID não viram separadores de caminho ou de consulta da URL por acidente. O ID é recebido separadamente e não é um campo de `AtualizacaoTarefa`.

Criar e depois atualizar um projeto

```
const projeto = await criarProjeto({
  nome: 'Estudo de Svelte',
  descricao: 'Exercícios da equipe',
  disciplina: 'Desenvolvimento Front-end',
  criadoEm: '2026-10-21'
});

const atualizado = await atualizarProjeto(projeto.id, {
  descricao: 'Exercícios e revisão da equipe'
});
```

Quem chama `criarProjeto` fornece `criadoEm`, junto com os outros campos exigidos. A função de serviço não inventa essa data; a API gera somente o ID. A atualização do exemplo altera apenas a descrição.

Por que existe o auxiliar `requisitar`?

As funções precisam fazer tarefas em comum: chamar `fetch`, verificar a resposta e ler o JSON. O auxiliar centraliza esses passos para não repeti-los em cada operação.

```
async function requisitar<T>(
  caminho: string,
  opcoes?: RequestInit
): Promise<T>
```

`RequestInit` é o tipo das opções aceitas pelo `fetch`, como `method`, `headers` e `body`. O `?` torna esse argumento opcional. As leituras podem chamar apenas com o caminho; as escritas passam suas opções.

```
return requisitar<Tarefa>(
  `/tarefas/${encodeURIComponent(id)}`,
  {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(dados)
  }
);
```

O `T` indica o tipo de resposta esperado por quem chama. Em `requisitar<Tarefa>`, esperamos uma tarefa; em `requisitar<Projeto>`, esperamos um projeto.

Como `ErroApi` ajuda?

Se a resposta HTTP não indicar sucesso, o auxiliar lança `ErroApi`, guardando o código em `status`. Uma tela pode distinguir, por exemplo, um registro não encontrado (`404`) de outra falha.

```
try {
  await atualizarTarefa('2', { prioridade: 'alta' });
} catch (erro) {
  if (erro instanceof ErroApi && erro.status === 404) {
    // Mostrar que a tarefa não foi encontrada.
  } else {
    // Mostrar que não foi possível salvar a alteração.
  }
}
```

Uma falha de conexão pode ocorrer antes de existir resposta HTTP. Nesse caso, o erro original do `fetch` também chega ao `catch`; não presumimos que todo erro seja um `ErroApi`.

Como chegar a esta parte no código: abra `src/lib/servicos/api.ts` e procure as quatro funções da tabela. Depois abra `src/tipos.ts` para ver seus tipos de entrada. Na tela de nova tarefa, procure a chamada de `criarTarefa`: ela é a ligação entre as issues #18 e #16.

08 / Issue #19: editar uma tarefa existente

Cadastro começa vazio e cria um registro. Edição começa com um registro já salvo e altera esse mesmo registro. Em `/tarefas/2/editar`, o ID `2` escolhe qual tarefa carregar.

1. Ler `page.params.id`.
2. Chamar `buscarTarefa(id)`, que faz GET.
3. Mostrar carregamento ou erro; quando chegar a tarefa, preencher `FormularioTarefa`.
4. Receber os dados pelo callback `salvar`.
5. Aguardar `atualizarTarefa(tarefa.id, dados)`, que faz PATCH, e navegar para o detalhe.

```
await atualizarTarefa(tarefa.id, dados);
await goto(`/tarefas/${tarefa.id}`);
```

Por que há um $effect?

A página observa o ID da URL. Se ele mudar, busca o novo registro. O efeito usa uma variável `ativo` e uma função de limpeza: uma resposta antiga não deve substituir os dados de outra tarefa depois da troca de rota.

`ErroApi` com status 404 vira “Tarefa não encontrada”. Outras falhas de busca recebem uma mensagem de carregamento malsucedido. A falha ao salvar usa outro estado, `erroEnvio`, e mantém o formulário disponível para correção ou nova tentativa.

Quem faz o quê?

O componente guarda os campos e valida. A página chama a API e controla `enviando`. O módulo da API monta o pedido HTTP. Essa separação permite estudar cada responsabilidade sem misturar tudo em uma função.

Como chegar: abra `src/routes/tarefas/[id]/editar/+page.svelte`. Procure `buscarTarefa`, `valorInicial={tarefa}` e `atualizarTarefa`. Ao demonstrar, altere um campo, salve e confira o mesmo ID no detalhe.

09 / Validar antes de enviar

A #20 trabalha em duas frentes: conferir o que a pessoa preencheu e impedir outro envio enquanto o primeiro está em andamento. Para isso, aproveitamos a validação do navegador e o auxiliar de acessibilidade que já veio do grupo.

Por que `required` sozinho não cobre tudo?

```
''      // vazio
'   '   // tem caracteres, mas só espaços
'Svelte' // tem texto
```

Em um campo de texto, `required` bloqueia o valor vazio. Um texto formado apenas por espaços pode passar por essa regra. A função `textoObrigatorioValido` acrescenta a conferência que faltava:

```
export function textoObrigatorioValido(valor: string): boolean {
  return valor.trim().length > 0;
}
```

`trim()` obtém o texto sem espaços em branco nas pontas. Se nada sobrar, o resultado tem tamanho zero e a função retorna `false`. Ela usa esse resultado para conferir o preenchimento; não modifica por conta própria o texto digitado.

Como essa regra chega ao formulário?

`validarObrigatorios` é uma *action* do Svelte: uma função ligada a um elemento por `use:`. Ela recebe o formulário quando ele é montado no navegador e acompanha os campos de texto.

```
<form
  onsubmit={aoEnviar}
  use:validarObrigatorios
  use:acessibilidadeFormulario
>
  ...
</form>
```

A nova action examina somente `input type="text"` e `textarea` com `required`. O seletor de projeto e o prazo continuam com suas regras nativas. A descrição é obrigatória no cadastro de nova tarefa; no formulário de edição, continua opcional conforme a entrega do colega.

`setCustomValidity`: adicionar uma regra ao navegador

```
campo.setCustomValidity(
  'Preencha este campo com texto, sem usar somente espaços.'
);

// Depois que essa regra for atendida:
campo.setCustomValidity('');
```

Uma mensagem não vazia marca um erro personalizado. Passar `''` limpa esse erro, permitindo ao navegador avaliar as outras regras. Não significa declarar que qualquer valor é válido.

O código deixa o valor realmente vazio para `required` tratar. A regra personalizada cuida do valor que contém caracteres, mas fica vazio depois de `trim()`. O formulário mantém a validação nativa do HTML, sem `novalidate`.

Por que o erro aparece perto do campo?

O auxiliar existente `acessibilidadeFormulario` escuta o evento `invalid`. Quando um campo é inválido, ele acrescenta uma mensagem de texto ao lado desse campo, marca `aria-invalid` e usa `aria-describedby` para relacionar campo e mensagem.

Assim, o erro continua disponível mesmo depois que o balão do navegador fecha. Quando o valor é corrigido e fica válido, o auxiliar remove sua mensagem e as marcações de erro. O navegador continua responsável por indicar o primeiro campo inválido durante a tentativa de envio.

Captura antes da propagação: por que a ordem importa?

```
formulario.addEventListener('input', aoAlterar, true);
formulario.addEventListener('change', aoAlterar, true);
```

O `true` pede para tratar esses eventos na fase de **captura**. Primeiro a nova regra atualiza a validade; depois, na fase de propagação, o auxiliar de acessibilidade lê o resultado. Isso evita que uma mensagem antiga permaneça depois de corrigir o texto.

O evento `invalid` não se propaga da mesma forma que `input`. O auxiliar de acessibilidade já o escuta em captura para atender aos campos do formulário. A #20 complementa essa estrutura existente.

Botão desabilitado e guarda no código

`disabled` comunica que o botão está indisponível e bloqueia seu uso normal. A função de envio também verifica o estado antes de chamar o código que grava a tarefa:

```
if (enviando || carregandoProjetos || erroProjetos) return;
if (!(evento.currentTarget as HTMLFormElement).reportValidity()) return;
```

O primeiro `if` impede executar o envio enquanto ele já está acontecendo ou sem ter os projetos disponíveis. O segundo pede a conferência nativa com `reportValidity()`; se houver campo inválido, a função encerra ali.

Durante a gravação, o botão mostra **Salvando...** e permanece desabilitado. A guarda no código dá proteção adicional contra chamadas repetidas; o cadastro também mantém seu controle de tarefa já criada para não repetir um POST depois do sucesso.

Erro de preenchimento e falha da API são diferentes

Situação	Como tratamos
Título vazio ou só com espaços	Impedir o envio e mostrar a mensagem junto do campo. A pessoa corrige o conteúdo.
Os projetos ainda estão carregando ou falharam	Não enviar com as opções indisponíveis. Mostrar a situação de carregamento ou erro.
A API falha ao salvar dados válidos	Manter os dados e mostrar a falha de gravação. Depois que o envio termina, permitir uma nova tentativa.

A mensagem de uma tentativa anterior de salvar não é, por si só, motivo para bloquear a próxima. Por isso, a guarda do componente verifica `erroProjetos`, mas não bloqueia o envio apenas porque a prop `erro` ainda contém a falha anterior da API.

Como chegar a esta parte: abra `src/lib/utils/validarObrigatorios.ts` para ler a regra. Depois procure `use:validarObrigatorios` na página de nova tarefa e em `FormularioTarefa.svelte`. Compare com `use:acessibilidadeFormulario` para separar quem confere o valor de quem apresenta o erro.

10 / Issue #21: entender e operar o formulário

Uma pessoa precisa reconhecer os campos, chegar até eles por teclado e compreender o erro. Só deixar uma borda vermelha não explica qual é o problema.

```
<label for="tarefa-titulo">Título (obrigatório)</label>
<input id="tarefa-titulo" type="text" required />
```

`for` aponta para o `id` do campo. O label dá um nome persistente ao controle. Placeholder sozinho não substitui esse nome.

Recurso	O que comunica
`aria-invalid="true"`	O campo está marcado como inválido.
`aria-describedby="..."`	Associa o campo ao texto explicativo por ID.
`role="alert"`	Marca uma mensagem importante, como o erro apresentado.
`aria-busy`	Informa que aquela região está em processamento.
`:focus-visible`	Mostra onde está o foco ao usar teclado.

O auxiliar do grupo

`acessibilidadeFormulario` acompanha eventos de validação nos formulários de tarefa. Ele cria uma mensagem ao lado do campo, associa essa mensagem por ARIA e a retira quando o campo fica válido. Sua limpeza também remove os ouvintes de eventos quando o formulário sai da tela.

Esse auxiliar não chama `focus()`. Nos formulários de tarefa, o foco no primeiro campo inválido vem do comportamento de validação do navegador. Nos projetos, o componente constrói seus próprios erros e associações ARIA.

Uma conferência por teclado

1. Use Tab e Shift+Tab para percorrer os campos, o botão e o cancelamento.
2. Confira se o foco é visível e segue uma ordem compreensível.
3. Preencha um seletor com as setas e tente enviar pelo teclado.
4. Deixe um campo obrigatório inválido e leia a mensagem escrita junto dele.
5. Corrija o valor e confira se a mensagem deixa de indicar um erro antigo.

Como chegar: abra `src/lib/utils/acessibilidadeFormulario.ts` e procure `mostrar`, `limpar` e os eventos `invalid`, `input` e `change`.

11 / Issue #22: cadastro e edição de projetos

Projeto e tarefa têm contratos diferentes. No formulário de projeto a pessoa preenche `nome`, `descricao` e `disciplina`. A API gera o `id`; a página de cadastro acrescenta `criadoEm`.

Cadastro em /projetos/novo

A página prepara os três campos vazios e passa ao componente `FormularioProjeto`. Sua função de salvar recebe os campos, acrescenta a data local e chama a API.

```
const projeto = await criarProjeto({
  ...dados,
  criadoEm: hojeLocal()
});
return `/projetos/${encodeURIComponent(projeto.id)}`;
```

O componente aguarda esse retorno e usa o endereço para continuar. `hojeLocal()` monta ano, mês e dia no fuso do computador. Isso evita usar sem querer o dia seguinte de UTC ao cadastrar à noite no Brasil.

Edição em /projetos/3/editar

A página carrega o projeto e passa seus valores atuais ao mesmo componente. Ao salvar, compara os campos com o registro carregado e monta `alteracoes` apenas com o que mudou.

```
if (dados.nome !== atual.nome) {
  alteracoes.nome = dados.nome;
}
if (Object.keys(alteracoes).length > 0) {
  await atualizarProjeto(atual.id, alteracoes);
}
```

O ID e a data de criação permanecem os do registro original. Se nada mudar, essa página não envia PATCH. O bloco `{#key projeto.id}` recria o formulário quando outro projeto é carregado; por isso, a cópia inicial dos campos pode ser feita uma vez.

Validação e envio têm uma implementação própria

`FormularioProjeto` usa `novalidate`, retira espaços das pontas e cria mensagens para campos vazios em sua função `validar()`. Seus controles continuam com `required`, mas `novalidate` desativa a verificação automática dessas regras no envio: `validar()` confere os três campos. Já `preventDefault()` impede o envio tradicional do formulário.

O `fieldset` é bloqueado enquanto `salvando` é verdadeiro. A guarda `salvando || salvo` evita um segundo envio durante a operação ou depois do sucesso. Se a API falhar, os dados são mantidos. Se a gravação der certo mas a navegação falhar, aparece um link “Continuar”.

Não misture as duas estratégias: os formulários de tarefa preservam a validação nativa e a complementam; o formulário de projeto valida por código com `novalidate`. ARIA comunica os resultados, mas não é uma função de validação.

Como chegar: abra as duas páginas em `src/routes/projetos`, o componente `FormularioProjeto.svelte` e `src/lib/utils/projetos.ts`.

12 / Issue #23: explicar a entrega em 5 minutos

O pedido da issue é mostrar o que foi implementado e fazer uma demonstração em, no máximo, cinco minutos. O arquivo da equipe está em docs/marco-3/apresentacao_marco3.pptx. O roteiro abaixo é uma sugestão de ensaio.

Tempo	O que mostrar ou dizer
0:00 a 0:35	Objetivo: sair da leitura para o cadastro e a edição, com SvelteKit e TypeScript.
0:35 a 1:15	Mapa de responsabilidades: página, formulário e API. Diferenciar POST e PATCH.
1:15 a 2:15	Abrir nova tarefa por um projeto, observar a seleção, preencher e salvar.
2:15 a 3:00	Editar a mesma tarefa e confirmar a mudança no detalhe.
3:00 a 3:45	Tentar um campo inválido, ler o erro e mostrar o foco pelo teclado.
3:45 a 4:35	Criar ou editar um projeto; explicar o reaproveitamento de FormularioProjeto.
4:35 a 5:00	Citar verificações e uma melhoria possível, deixando margem para encerrar.

Uma fala que você consegue defender

“A página identifica se vamos criar ou editar. Os campos ficam em estado reativo. Quando o preenchimento é válido, chamamos uma função tipada da API e esperamos a confirmação. Durante o envio bloqueamos novas tentativas. Se falhar, mostramos uma mensagem e mantemos os dados.”

Compare os slides com o código antes de apresentar

O slide sobre formulário fala do mesmo componente em criação e edição. Essa é a intenção de reutilização da #17; a nova tarefa ainda usa formulário próprio. O trecho de botão nos slides também é simplificado: o componente atual bloqueia por envio, carregamento e erro ao carregar projetos. Ao responder, use o código como referência.

Ensaio: explique em voz alta sem decorar cada linha. Quando falar “callback”, aponte `onenviar={salvar}`. Quando falar “PATCH”, aponte `atualizarTarefa`. Quando falar “acessível”, navegue por teclado e mostre o texto de erro.

13 / Como abrir pelo terminal

Esta entrega usa uma cópia de trabalho separada. A raiz dessa cópia é:

```
/Users/andreybuenoisoton/Documents/Codex/2026-10-14/create-a-new-site-with-sites-4/outputs/marco3-issue18
```

Abra essa pasta no VS Code. No terminal integrado, use `pwd` para confirmar onde você está. Todos os comandos abaixo partem da raiz do projeto, onde está `package.json`.

1. Instalar as dependências, quando necessário

```
npm ci
```

Instala as versões registradas no `package-lock.json`. É necessário em uma cópia recém-baixada ou quando as dependências precisam ser reinstaladas. O projeto pede Node.js 24 ou superior; `node --version` mostra sua versão.

2. Abrir a API em uma aba do terminal

```
npm run api
```

Inicia o `json-server` usando o `db.json` dessa pasta. O endereço esperado pela aplicação é `http://localhost:3000`. Esse terminal continua ocupado atendendo à API.

3. Abrir o site em outra aba do terminal

```
npm run dev
```

Inicia o servidor de desenvolvimento. Abra o endereço **Local** que ele imprimir, normalmente `http://localhost:5173`, e acrescente `/tarefas/nova`.

Não copie o prompt. Um trecho como `andrey@Mac projeto-g2-frontend %` apenas mostra o usuário e a pasta atual. Você digita somente o comando depois do `%`.

Com um servidor rodando, aquela aba não está pronta para comandos Git. Abra outra aba. Para encerrar um servidor que você iniciou, use **Ctrl+C** na aba dele.

O site e a API são dois programas. A porta **5173** mostra a interface; a porta **3000** responde com os dados. Abrir a API diretamente no navegador pode mostrar JSON, o que é esperado.

Conferir os tipos e preparar a aplicação

```
npm run check
npm run build
```

O primeiro procura problemas de tipos e de Svelte. O segundo gera a versão de produção. Eles ajudam a conferir o código, mas não substituem experimentar o formulário no navegador.

Executar os testes e conferir os contratos

```
node --test tests/*.test.mjs
npx tsc -p tests/tsconfig.json
```

Execute na raiz do projeto. O primeiro comando usa o executor de testes do Node em todos os arquivos `.test.mjs` da pasta: inclui a API, os auxiliares do grupo e a nova regra de texto

obrigatório. Os testes de integração da API usam um json-server com banco em memória, sem tocar no `db.json` da equipe.

O segundo comando executa o TypeScript para conferir entradas e retornos: verifica também que ID e campos inválidos sejam recusados pelos tipos. Leia os resultados impressos no terminal. A regra de texto e o comportamento visual dos erros também devem ser conferidos no navegador.

14 / O que você deve experimentar

Use esta lista como roteiro de estudo. As caixas são suas marcações locais e não representam um relatório automático de testes.

- Abrir `/tarefas/nova` e identificar a tela de cadastro.

- Abrir um projeto e usar seu link de nova tarefa. Conferir qual opção já vem selecionada.

- Trocar o projeto manualmente e perceber que sua escolha é mantida.

- Preencher todos os campos e salvar uma tarefa de teste. Conferir se ela aparece na lista.

- Observar o botão durante o envio e explicar por que ele fica desabilitado.

- Abrir um detalhe como `/tarefas/2` e perceber que continua sendo outra página.

- Tentar salvar um título só com espaços. Conferir a mensagem junto ao título e o bloqueio do envio.

- Corrigir o título e observar a mensagem de erro desaparecer quando o campo fica válido.

- Conferir que descrição vazia é recusada no cadastro, enquanto continua opcional na edição.

- Depois de uma falha de gravação, tentar novamente e conferir que a mensagem anterior não bloqueia a nova tentativa.

Verificação final da #20

Passaram 22 testes automatizados, a verificação dos tipos TypeScript, a checagem do Svelte sem erros nem avisos e a compilação. No navegador, conferimos campos vazios, texto só com espaços, erro junto ao campo, foco no primeiro inválido e limpeza da mensagem ao corrigir. Também conferimos cadastro e edição com resposta lenta, botão bloqueado, tentativa após erro da API e clique duplo com apenas um envio válido. A descrição opcional da edição foi preservada. Nenhum pedido foi enviado nas tentativas inválidas, e o banco original permaneceu intacto.

Verificações anteriores da tela e da API

Na verificação inicial da #18, o Svelte encontrou zero erros e avisos e a compilação passou. No navegador, conferimos cadastro, projeto pré-selecionado, troca manual, campos obrigatórios, clique duplo e recuperação após falha de conexão, usando uma cópia do banco.

Na entrega da #16, passaram 18 testes automatizados, a verificação dos contratos TypeScript, a checagem do Svelte sem erros nem avisos e a compilação. Foram conferidos POST e PATCH, ID no caminho, JSON no corpo, preservação de campos, erros HTTP e leituras existentes. Os testes de integração usaram banco em memória e encerraram seu próprio servidor. Nenhuma tela foi modificada nesta etapa da #16.

Criar ou atualizar registros grava dados na API em uso. Para estudar sem alterar os dados da equipe, faça os testes em uma cópia do banco.

Roteiro do Marco 3 completo

- **#16:** mostrar uma chamada de criação e uma de atualização; localizar os tipos.
- **#17 e #18:** localizar o componente e a tela nova; explicar a diferença de integração atual.
- **#19:** conferir os dados iniciais e a alteração do mesmo registro.
- **#20 e #21:** conferir campo inválido, mensagem textual, teclado e bloqueio de envio.
- **#22:** criar e editar projeto; conferir nome, descrição, disciplina, ID e data preservada.
- **#23:** ensaiar a apresentação com demonstração em até cinco minutos.

As verificações acima são um roteiro para você executar. Os resultados de testes registrados anteriormente nesta seção se referem às entregas anteriores; ampliar o material de estudo não executa novamente o aplicativo.

15 / Perguntas que a professora pode fazer

Estas são possibilidades de estudo, não uma previsão da prova. Treine explicar o motivo e apontar um exemplo no código. No final há um simulado de múltipla escolha para você responder.

1. Por que /tarefas/nova não abre o detalhe de uma tarefa chamada “nova”?

Porque existe uma rota fixa em `tarefas/nova/+page.svelte`. O SvelteKit dá prioridade à rota fixa sobre `tarefas/[id]/+page.svelte`.

2. De onde vem o projeto que aparece selecionado?

Do parâmetro `projetoId` na consulta da URL, enviado pelo link da página do projeto. O código lê essa sugestão e confere se ela corresponde a um projeto existente nas opções da API.

3. Qual é a diferença entre GET e POST nesta tela?

`GET` lê os projetos. `POST` envia os dados para criar a tarefa. A requisição de criação leva um corpo JSON.

4. Por que não enviamos um id junto com os campos?

O ID é gerado pela API. O tipo `NovaTarefa` usa os campos de `Tarefa` sem `id`, deixando isso claro enquanto programamos.

5. O que aconteceria se voltássemos para a lista antes de esperar a API?

A pessoa poderia ver a lista antes da gravação terminar ou sair do formulário sem perceber uma falha. A ordem correta é confirmar a criação e depois navegar.

6. O que bind:value faz quando você digita um título?

Atualiza o estado ligado ao campo. A ligação também funciona no outro sentido: quando o código altera esse estado, o valor mostrado no campo acompanha a mudança.

7. Para mudar só a prioridade, uso criarTarefa ou atualizarTarefa?

Uso `atualizarTarefa`, passando o ID do registro e um objeto como `{ prioridade: 'alta' }`. O pedido usa PATCH. `criarTarefa` usa POST para acrescentar uma nova tarefa.

8. Qual é a diferença entre Omit e Partial nesses tipos?

`Omit<Tarefa, 'id'>` retira o ID do tipo de criação. `Partial<NovaTarefa>` torna opcionais os campos da atualização, permitindo enviar apenas aqueles que devem mudar.

9. No cadastro de um projeto, quem fornece criadoEm?

O código que chama `criarProjeto` fornece esse campo no objeto `NovoProjeto`. A função envia os dados recebidos; a API gera o ID, sem inventar a data de criação.

10. Onde vão o ID e os novos valores em uma atualização?

O ID identifica o registro na URL, como `/tarefas/2`. Os valores que devem mudar vão no corpo JSON do PATCH. A função codifica o ID com `encodeURIComponent` ao montar o endereço.

11. Promise<Projeto> quer dizer que já tenho um projeto pronto?

Não. É um resultado que pode chegar depois. Ao usar `await`, esperamos a operação; se ela tiver sucesso, recebemos o projeto. Se falhar, tratamos o erro no `catch`.

12. Por que todas as funções usam requisitar?

Porque a chamada de rede, a leitura do JSON e a checagem de sucesso são passos comuns. O auxiliar recebe o caminho e opções `RequestInit`, evitando repetir esses passos em cada função.

13. Por que um título só com espaços precisa de outra regra além de required?

Porque ele contém caracteres, embora não tenha texto útil. A nova regra confere se ainda sobra algum conteúdo depois de `trim()`. Se não sobrar, marca um erro personalizado.

14. O que significa chamar setCustomValidity com uma string vazia?

Significa limpar o erro personalizado dessa regra. As outras verificações nativas, como `required`, continuam valendo. Isso permite que um campo corrigido volte a ser considerado válido.

15. Quem mostra o erro junto do campo?

O auxiliar `acessibilidadeFormulario`, já entregue pelo grupo. Ele usa o evento `invalid`, insere a mensagem em texto e relaciona a mensagem ao campo com atributos ARIA. A nova action fornece a regra adicional de validade para texto só com espaços.

16. Por que a nova regra escuta input e change em captura?

Para atualizar a validade antes que o auxiliar de acessibilidade, na fase de propagação desses eventos, leia o resultado. Assim, corrigir o texto também atualiza a mensagem na ordem certa.

17. Se o botão já está disabled, para que verificar enviando dentro da função?

O botão desabilitado bloqueia sua interação normal e comunica o estado. A função também verifica se há envio em andamento para evitar executar novamente o código de gravação. São duas proteções trabalhando juntas.

18. Uma mensagem de falha da API deve impedir a próxima tentativa de salvar?

Não. Depois que a tentativa termina, a pessoa precisa poder tentar outra vez. O código bloqueia durante o envio e quando os projetos estão indisponíveis; ele não usa a mensagem de uma falha anterior de gravação como trava permanente.

Mais perguntas para a apresentação

Qual é a diferença entre Svelte, SvelteKit e TypeScript?

Svelte organiza componentes e reatividade. O projeto usa SvelteKit para a estrutura de páginas e a navegação por arquivos. TypeScript descreve os tipos e ajuda a detectar incompatibilidades durante o desenvolvimento; seus tipos não validam sozinhos a resposta recebida da API.

Como o componente avisa à página que deve salvar?

A página passa a função salvar em onenviar. O componente chama onenviar(dados). Esse callback liga o formulário à decisão de gravar na API, sem obrigar o componente a conhecer a rota de destino.

O formulário de tarefa está mesmo reaproveitado nas duas telas?

Na base consultada, FormularioTarefa é usado na edição. A tela de nova tarefa tem formulário próprio. O componente foi preparado para reutilização, mas a integração nas duas telas ainda é uma melhoria possível. Em projetos, FormularioProjeto é usado de fato nos dois fluxos.

Por que buscar os dados antes de editar?

A pessoa precisa ver o estado atual do registro e alterar o mesmo ID. GET carrega a tarefa; PATCH grava as alterações. Abrir a edição sem os valores atuais pode levar a sobrescrever campos sem perceber.

Por que não basta deixar o erro vermelho?

Cor sozinha não informa qual campo está errado nem atende quem não a distingue. O projeto usa texto junto ao campo, labels e associações ARIA, além de foco visível e controles operáveis por teclado.

O projeto novo e o editado mudam a data de criação do mesmo jeito?

Não. O cadastro acrescenta criadoEm com hojeLocal(). A edição preserva criadoEm e id e envia apenas os campos alterados. A data de criação não deve virar a data de cada edição.

Todos os formulários validam do mesmo jeito?

Não. Tarefas complementam a validação nativa com a regra de espaços e o auxiliar de acessibilidade. FormularioProjeto usa novalidate e verifica seus campos em validar(). É preciso explicar cada implementação como ela realmente é.

Se fosse melhorar a solução, por onde começaria?

Uma melhoria ligada ao próprio pedido é integrar FormularioTarefa também à nova tarefa, preservando a seleção de projeto, as mensagens e os testes. Outra é revisar os slides e o README para que descrevam o código atual. Isso é uma proposta, não algo feito por este guia.

Para consultar quando precisar

- SvelteKit: páginas e rotas
- SvelteKit: prioridade entre rotas
- SvelteKit: informações da página atual
- SvelteKit: navegar com goto
- SvelteKit: preservação da página ao navegar
- Svelte: estado com $state
- Svelte: ligação com bind
- Svelte: actions ligadas a elementos com use:

- HTML: validação nativa e setCustomValidity
- TypeScript: Partial
- TypeScript: Omit
- HTTP: especificação do método PATCH
- Svelte: props e entradas de componentes
- Svelte: leitura sem acompanhamento com untrack
- Requisitos: issues do Marco 3 no GitHub

17 / Simulado de múltipla escolha

Marque uma alternativa por questão. Tente responder sem olhar o gabarito comentado, que começa após as 24 questões. Na versão HTML, você pode marcar e corrigir automaticamente.

Questão 1 · Issue #16

A pessoa preencheu o cadastro de uma tarefa e clicou em Salvar. Qual operação a função criarTarefa faz na API?

A) GET /tarefas, pois GET grava um cadastro novo.

B) PATCH /tarefas/1, sempre alterando a tarefa 1.

C) DELETE /tarefas, antes de criar a tarefa.

D) POST /tarefas, enviando os dados do cadastro.

Questão 2 · Issue #16

No arquivo de tipos, por que NovaTarefa usa Omit<Tarefa, 'id'> e AtualizacaoTarefa usa Partial<NovaTarefa>?

A) Para criar sem fornecer id e permitir uma atualização com apenas parte dos campos.

B) Para converter automaticamente todos os textos em números.

C) Para obrigar cada atualização a criar um novo id.

D) Para executar a validação do banco de dados em tempo de execução.

Questão 3 · Issue #16

Para que servem JSON.stringify(dados) e o cabeçalho Content-Type: application/json nas funções de escrita?

A) Para escolher qual página Svelte será aberta.

B) Para transformar o objeto em texto JSON e informar o formato enviado à API.

C) Para criar o arquivo tipos.ts no servidor.

D) Para transformar a API em uma página com botões.

Questão 4 · Issue #16

Por que o auxiliar requisitar verifica resposta.ok antes de devolver os dados?

A) Porque fetch lança um erro automaticamente em toda resposta 404.

B) Porque resposta.ok valida todos os campos do formulário.

C) Para tratar respostas HTTP de erro, como 404 e 500, lançando ErroApi com o status.

D) Porque toda resposta que chega pela rede já representa sucesso.

Questão 5 · Issue #17

No FormularioTarefa, o que bind:value={dados.titulo} faz?

A) Faz um POST a cada letra digitada.

B) Busca o título no parâmetro id da URL.

C) Muda o nome da propriedade titulo para value na API.

D) Mantém o campo e dados.titulo ligados: digitar atualiza o estado, e mudar o estado atualiza o campo.

Questão 6 · Issue #17

Qual combinação descreve corretamente as runas usadas no formulário?

A) $props cria uma requisição e $state cria uma rota.

B) $props salva no json-server; $state salva no GitHub.

C) $props recebe valores do componente pai; $state mantém dados reativos da interface.

D) $props e $state substituem os tipos TypeScript.

Questão 7 · Issue #17

Na edição de tarefa, por que FormularioTarefa recebe onenviar={salvar}?

A) Para receber o endereço do servidor como uma string.

B) Para o formulário editar automaticamente todos os arquivos da aplicação.

C) Para executar salvar assim que a página for importada.

D) Para entregar os dados preenchidos à função da página, que decide como gravar.

Questão 8 · Issue #18

Qual é a diferença entre /tarefas/nova e /tarefas/2?

A) /tarefas/nova é a página fixa de cadastro; em /tarefas/2, o 2 preenche o parâmetro [id].

B) nova e 2 são nomes de variáveis TypeScript criadas no formulário.

C) As duas URLs sempre mostram o detalhe da tarefa de id nova.

D) Uma URL com texto nunca pode ser uma rota fixa no SvelteKit.

Questão 9 · Issue #18

Ao abrir /tarefas/nova?projetoId=3, como o projeto 3 é sugerido no cadastro?

A) A página lê searchParams, confere se o projeto existe na lista e então o seleciona.

B) O id 3 vira automaticamente o id da tarefa nova.

C) A página cria o projeto 3 antes de mostrar o formulário.

D) O parâmetro dispensa a consulta dos projetos e a validação da escolha.

Questão 10 · Issue #18

Se a API falhar ao salvar uma tarefa nova, o que a tela implementada faz?

A) Apaga os campos e retorna para a lista como se tivesse salvo.

B) Inventa um id e informa que a tarefa foi criada.

C) Atualiza o Git para recuperar a conexão com a API.

D) Mostra uma mensagem, mantém os dados e permite tentar novamente.

Questão 11 · Issue #19

Qual sequência representa a edição de uma tarefa no projeto?

A) Apagar a tarefa antiga e abrir um formulário sempre vazio.

B) Mostrar apenas o id, sem consultar os dados da tarefa.

C) Ler o id da URL, buscar a tarefa, preencher o formulário e enviar a atualização ao salvar.

D) Criar uma tarefa nova com POST toda vez que a pessoa clicar em editar.

Questão 12 · Issue #19

Por que atualizarTarefa recebe o id separadamente dos dados do formulário?

A) Porque o id deve mudar quando o título muda.

B) Porque PATCH não aceita nenhum campo no corpo da requisição.

C) Porque o id identifica o recurso na URL, enquanto o corpo leva os campos que serão atualizados.

D) Porque o id só existe dentro do CSS do formulário.

Questão 13 · Issue #19

Na busca da tarefa para edição, para que serve a variável ativo que vira false na limpeza do $effect?

A) Para impedir que uma resposta antiga atualize a tela depois de a busca perder a validade.

B) Para cancelar fisicamente a requisição de rede usando AbortController.

C) Para salvar automaticamente a tarefa enquanto ela carrega.

D) Para informar à API que a tarefa está em andamento.

Questão 14 · Issue #20

Por que o projeto acrescenta uma validação para textos que contêm somente espaços?

A) Porque required já remove todos os espaços automaticamente.

B) Porque todo espaço deve ser removido do meio dos nomes completos.

C) Porque required pode aceitar um texto com espaços, mas trim().length permite detectar que não há conteúdo.

D) Porque TypeScript rejeita automaticamente textos em branco ao executar o site.

Questão 15 · Issue #20

O formulário de nova tarefa desabilita os controles e verifica salvando antes de enviar. Qual problema isso ajuda a evitar?

A) Que cliques repetidos iniciem cadastros duplicados enquanto um envio já está acontecendo.

B) Que o navegador mostre o nome do projeto.

C) Que a API devolva qualquer mensagem de erro.

D) Que outra pessoa consulte a lista de tarefas.

Questão 16 · Issue #20

Qual teste verifica a validação de forma mais útil para quem usa a aplicação?

A) Tentar enviar vazio ou com espaços, corrigir os campos e verificar que o cadastro válido é salvo uma vez.

B) Conferir só se a borda do botão é azul.

C) Abrir apenas a página inicial e encerrar o teste.

D) Considerar que a aplicação está correta porque o arquivo foi salvo.

Questão 17 · Issue #21

Qual é o papel de <label for="titulo"> junto de um input com id="titulo"?

A) Definir qual id a API vai gerar para a tarefa.

B) Associar o texto ao campo, dando um nome acessível e permitindo focá-lo ao clicar no rótulo.

C) Transformar o campo em uma rota dinâmica.

D) Enviar o formulário automaticamente quando o rótulo for lido.

Questão 18 · Issue #21

Quando um campo está inválido, por que usar aria-invalid e aria-describedby com uma mensagem de erro?

A) Para guardar o erro permanentemente no banco de dados.

B) Para substituir required em qualquer situação, sem validar os dados.

C) Para esconder o erro de quem navega com teclado.

D) Para informar o estado inválido e relacionar o campo à explicação do erro nas tecnologias assistivas.

Questão 19 · Issue #21

Qual verificação ajuda a avaliar se o cadastro pode ser usado sem mouse?

A) Remover os rótulos e deixar somente as cores dos campos.

B) Percorrer com Tab e Shift+Tab, verificar o foco visível e acionar o envio pelo teclado.

C) Desativar o foco para a página parecer mais limpa.

D) Conferir apenas se o cursor muda quando passa sobre o botão.

Questão 20 · Issue #22

Como a data criadoEm é definida no cadastro de projeto implementado?

A) A pessoa precisa digitar manualmente um timestamp em UTC.

B) A página chama hojeLocal() para usar a data do computador no formato AAAA-MM-DD.

C) Ela é alterada para amanhã sempre que o projeto for editado.

D) O valor é copiado do prazo da primeira tarefa do projeto.

Questão 21 · Issue #22

Na edição de projeto, o usuário mudou apenas a disciplina. Qual comportamento está implementado?

A) Apagar o projeto e cadastrar outro com um id diferente.

B) Enviar no PATCH a disciplina alterada, preservando id e criadoEm.

C) Atualizar criadoEm para a data da edição e remover as tarefas.

D) Enviar um POST sem verificar quais campos mudaram.

Questão 22 · Issue #22

Por que as páginas de novo projeto e editar projeto usam o mesmo FormularioProjeto?

A) Porque todas as páginas precisam ter exatamente a mesma URL.

B) Para fazer o cadastro e a edição usarem sempre POST.

C) Para compartilhar campos e validação, recebendo valores iniciais e uma função de gravação de cada página.

D) Para impedir que a página de edição carregue dados existentes.

Questão 23 · Issue #23

Em uma apresentação de cinco minutos, qual demonstração comunica melhor o objetivo do Marco 3?

A) Ler todas as linhas de CSS, sem executar a aplicação.

B) Mostrar somente o quadro de issues e dizer que tudo funciona.

C) Passar a maior parte do tempo instalando o Node na frente da turma.

D) Demonstrar cadastro, validação de um erro e edição, explicando o caminho tela -> API -> resposta.

Questão 24 · Issue #23

Se a professora perguntar 'o que garante que isso funciona?', qual resposta descreve evidências adequadas?

A) O push terminou, então qualquer comportamento da aplicação está garantido.

B) Conferimos os tipos e o build e testamos cadastro, edição, validação e falha da API, observando os resultados.

C) TypeScript impede toda falha de rede e todo dado inválido em execução.

D) A cor do botão confirmou que a gravação chegou ao servidor.

Gabarito comentado

Confira depois de responder. Cada acerto vale um ponto; o total é 24. Volte ao trecho indicado para revisar o motivo, em vez de decorar apenas a letra.

1. Resposta D · Issue #16

POST /tarefas, enviando os dados do cadastro.

POST cria um recurso na coleção /tarefas. GET apenas consulta. PATCH altera um recurso existente. O json-server gera o id do cadastro novo.

2. Resposta A · Issue #16

Para criar sem fornecer id e permitir uma atualização com apenas parte dos campos.

Omit remove id do tipo usado para cadastrar. Partial torna opcionais as propriedades do tipo usado no PATCH. Esses tipos ajudam durante o desenvolvimento; não validam sozinhos uma resposta real da API.

3. Resposta B · Issue #16

Para transformar o objeto em texto JSON e informar o formato enviado à API.

O body da requisição leva o JSON como texto. JSON.stringify faz essa conversão, e Content-Type informa à API que o corpo está em JSON.

4. Resposta C · Issue #16

Para tratar respostas HTTP de erro, como 404 e 500, lançando ErroApi com o status.

fetch pode resolver a Promise mesmo quando o servidor devolve 404 ou 500. O código precisa conferir resposta.ok. Aqui, ErroApi guarda o status para a tela distinguir situações como item não encontrado.

5. Resposta D · Issue #17

Mantém o campo e dados.titulo ligados: digitar atualiza o estado, e mudar o estado atualiza o campo.

bind:value conecta o valor exibido pelo input à propriedade reativa. Digitar muda dados.titulo; isso ainda não grava a tarefa. A gravação ocorre no envio do formulário.

6. Resposta C · Issue #17

$props recebe valores do componente pai; $state mantém dados reativos da interface.

O componente pai passa props como valorInicial e textoEnvio. O formulário usa $props para recebê-las e $state para acompanhar valores que mudam durante o preenchimento.

7. Resposta D · Issue #17

Para entregar os dados preenchidos à função da página, que decide como gravar.

onenviar é uma função recebida por prop, também chamada de callback. O formulário reúne os dados e chama essa função. Na página de edição, salvar usa atualizarTarefa e depois navega para o detalhe.

8. Resposta A · Issue #18

/tarefas/nova é a página fixa de cadastro; em /tarefas/2, o 2 preenche o parâmetro [id].

A pasta tarefas/nova tem seu próprio +page.svelte. A pasta tarefas/[id] atende os detalhes por id. Rotas fixas têm prioridade sobre parâmetros dinâmicos, então nova abre o cadastro.

9. Resposta A · Issue #18

A página lê searchParams, confere se o projeto existe na lista e então o seleciona.

projetoId=3 é um parâmetro de consulta da URL, lido por page.url.searchParams.get('projetoId'). A página verifica a lista carregada antes de selecionar. Um id inexistente gera um aviso e permite escolher outro projeto.

10. Resposta D · Issue #18

Mostra uma mensagem, mantém os dados e permite tentar novamente.

O catch mostra erroSalvar. Os dados continuam no estado do formulário, evitando que a pessoa precise preencher tudo de novo. A navegação de sucesso acontece depois de o cadastro ser confirmado.

11. Resposta C · Issue #19

Ler o id da URL, buscar a tarefa, preencher o formulário e enviar a atualização ao salvar.

A página lê page.params.id e chama buscarTarefa. Após a resposta, passa a tarefa como valorInicial para FormularioTarefa. O envio chama atualizarTarefa, que faz PATCH no id existente.

12. Resposta C · Issue #19

Porque o id identifica o recurso na URL, enquanto o corpo leva os campos que serão atualizados.

O endpoint /tarefas/{id} identifica qual tarefa será alterada. O formulário monta os campos sem incluir id no corpo. O tipo AtualizacaoTarefa também permite atualizar campos sem trocar o identificador.

13. Resposta A · Issue #19

Para impedir que uma resposta antiga atualize a tela depois de a busca perder a validade.

Se o id mudar ou o efeito for encerrado, ativo fica false. Os callbacks conferem esse valor antes de atualizar o estado. Isso ignora respostas antigas; a variável, sozinha, não cancela o tráfego da requisição.

14. Resposta C · Issue #20

Porque required pode aceitar um texto com espaços, mas trim().length permite detectar que não há conteúdo.

Um valor como ' ' não está vazio para a validação nativa de um input de texto. O auxiliar verifica valor.trim().length > 0 e cria um erro quando um campo obrigatório contém apenas espaços. Isso não proíbe espaços entre as palavras.

15. Resposta A · Issue #20

Que cliques repetidos iniciem cadastros duplicados enquanto um envio já está acontecendo.

A interface mostra Salvando... e bloqueia novos envios durante a requisição. A verificação no handler reforça esse controle. Na criação há também tarefaCriada, para não cadastrar de novo caso só a navegação depois do POST falhe.

16. Resposta A · Issue #20

Tentar enviar vazio ou com espaços, corrigir os campos e verificar que o cadastro válido é salvo uma vez.

Esse teste acompanha o comportamento esperado: impedir dados inválidos, mostrar os erros, permitir a correção e aceitar um envio válido. Conferir tipos e build ajuda, mas não substitui a verificação do fluxo de uso.

17. Resposta B · Issue #21

Associar o texto ao campo, dando um nome acessível e permitindo focá-lo ao clicar no rótulo.

for e id conectam o rótulo ao controle. Essa relação ajuda pessoas que usam leitores de tela e também melhora a interação com mouse ou toque. Um placeholder sozinho não oferece a mesma identificação persistente.

18. Resposta D · Issue #21

Para informar o estado inválido e relacionar o campo à explicação do erro nas tecnologias assistivas.

aria-invalid informa que há um problema. aria-describedby referencia o id do texto que explica o problema. O código deve manter esses atributos coerentes com a validação e limpar o erro depois da correção.

19. Resposta B · Issue #21

Percorrer com Tab e Shift+Tab, verificar o foco visível e acionar o envio pelo teclado.

A navegação deve alcançar os campos e ações em uma ordem compreensível. O foco visível mostra onde a pessoa está. Teclado, mensagens e associações de rótulos precisam funcionar em conjunto.

20. Resposta B · Issue #22

A página chama hojeLocal() para usar a data do computador no formato AAAA-MM-DD.

O cadastro acrescenta criadoEm: hojeLocal() ao criar o projeto. A função monta ano, mês e dia no fuso local. Usar diretamente toISOString() usaria UTC, o que pode adiantar o dia em relação ao horário do Brasil à noite.

21. Resposta B · Issue #22

Enviar no PATCH a disciplina alterada, preservando id e criadoEm.

A página compara nome, descricao e disciplina com o projeto carregado e monta alteracoes. Faz PATCH somente se houver diferenças. id identifica o projeto existente e criadoEm continua representando sua criação.

22. Resposta C · Issue #22

Para compartilhar campos e validação, recebendo valores iniciais e uma função de gravação de cada página.

O componente recebe inicial, rótulos e aoSalvar. A página de criação fornece uma função que faz POST. A de edição fornece uma função que calcula alterações e faz PATCH. Assim, o formulário pode ser reaproveitado sem decidir sozinho qual operação executar.

23. Resposta D · Issue #23

Demonstrar cadastro, validação de um erro e edição, explicando o caminho tela -> API -> resposta.

Um fluxo curto e preparado mostra o resultado e permite explicar os conceitos: formulário e estado, validação, POST/PATCH, retorno da API e navegação. Separe o tempo entre problema, demonstração e explicação do código principal.

24. Resposta B · Issue #23

Conferimos os tipos e o build e testamos cadastro, edição, validação e falha da API, observando os resultados.

Cada verificação cobre um aspecto: tipos detectam incompatibilidades; build verifica a compilação; testes dos fluxos verificam o comportamento e as respostas de erro. Nenhuma checagem isolada garante que todo problema possível foi eliminado.